

ТЕОРИЯ ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ

Каганов Ю.Т.

к.т.н., доцент каф. ИУ-9

**«Теоретическая информатика и компьютерные
технологии»**

ЛЕКЦИЯ № 4

**Методы оптимизации в обучении
нейронных сетей**

Постановка задачи многомерной оптимизации

Рассмотрим задачи нелинейной оптимизации в общем случае.

Пусть $x = (x^{(1)}, \dots, x^{(n)})^T \in R^n$, S – некоторое множество из пространства R^n . Пусть $f_0(x), \dots, f_m(x): S \rightarrow R^1$. Тогда задачу минимизации можно рассматривать в следующем виде

$$\min f_0(x) \text{ при } f_j(x) \leq 0, j = 1, \dots, m, x \in S. \quad (1)$$

$f_0(x)$ - будем называть *целевой функцией*. Функции $f_j(x) \leq 0$, как было принято нами ранее, обозначим $g_j(x) \leq 0$ или $g_j(x) = 0$.

Они называются *функциями ограничений* и зависят от типа ограничений – равенств или неравенств. Множество S –

называется *базовым допустимым множеством*, а множество Q

$Q = (x \in S | g_j(x) \leq 0, j=1, \dots, m)$. Называется просто *допустимым множеством*.

Классификация типов задач многомерной ОПТИМИЗАЦИИ

- *Задачи на условный экстремум:* $Q \subset R^n$.
- *Задачи на безусловный экстремум:* $Q \equiv R^n$.
- *Гладкие задачи:* $\forall f_j(x) \in C^k(Q), j = 0, \dots, m$.
- *Негладкие задачи:* $\exists f_j(x) \notin C^k(Q), j = 0, \dots, m; \forall k \geq 1$.
- *Задачи с линейными ограничениями:* все функциональные ограничения являются *линейными функциями*:

$$f_j(x) = \sum_{i=1}^n a_j^i x^i + b_j, j = 1, \dots, m. \text{ При этом}$$

*базовое множество S является многогранником. Если $f_0(x)$ также линейная функция, то исходная задача (1) называется **задачей линейной оптимизации**.*

Классификация типов задач многомерной оптимизации

1. Свойства допустимых множеств.

- Задача (1) называется *допустимой*, если $Q \neq \emptyset$.
- Задача (1) называется *строго допустимой*, если существует такой вектор $x \in \text{int}Q$, что $f_j(x) < 0$ (или > 0) для всех ограничений-неравенств и $f_j(x) = 0$ для всех ограничений-равенств (условие Слэйтера).

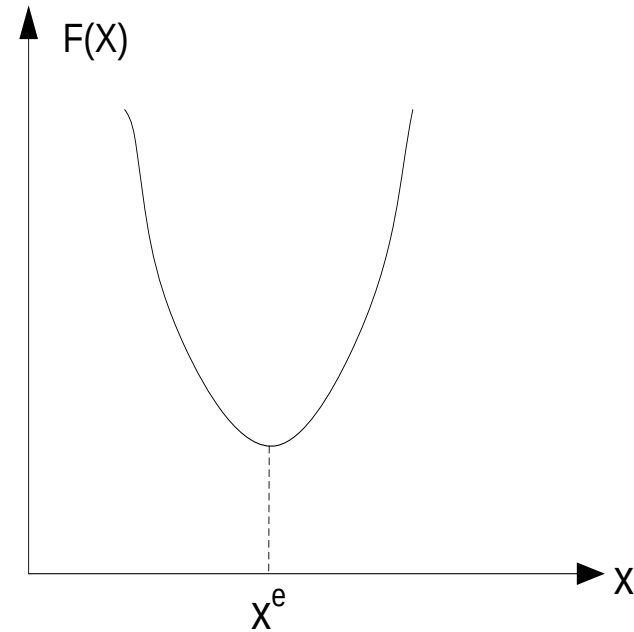
2. Типы решений задачи (1).

- Точка x^* называется оптимальным глобальным решением задачи (1), если $f_0(x^*) \leq f_0(x)$ для всех $x \in Q$ (*глобальный минимум*). В этом случае $f_0(x^*)$ называется (*глобальным*) оптимальным значением задачи.
- точка x^* называется *локальным решением задачи (1)*, если для всех $x \in \bar{Q} \subset Q$ выполнено неравенство $f_0(x^*) \leq f_0(x)$ (*локальный минимум*).

Один экстремум

Виды задач:

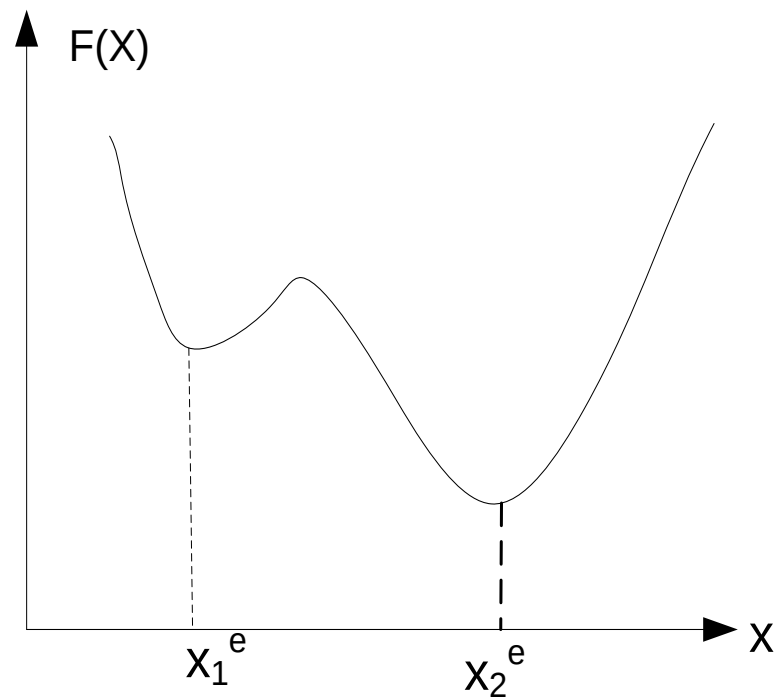
- методы **одноэкстремальной** оптимизации (единственный экстремум — унимодальная функция);



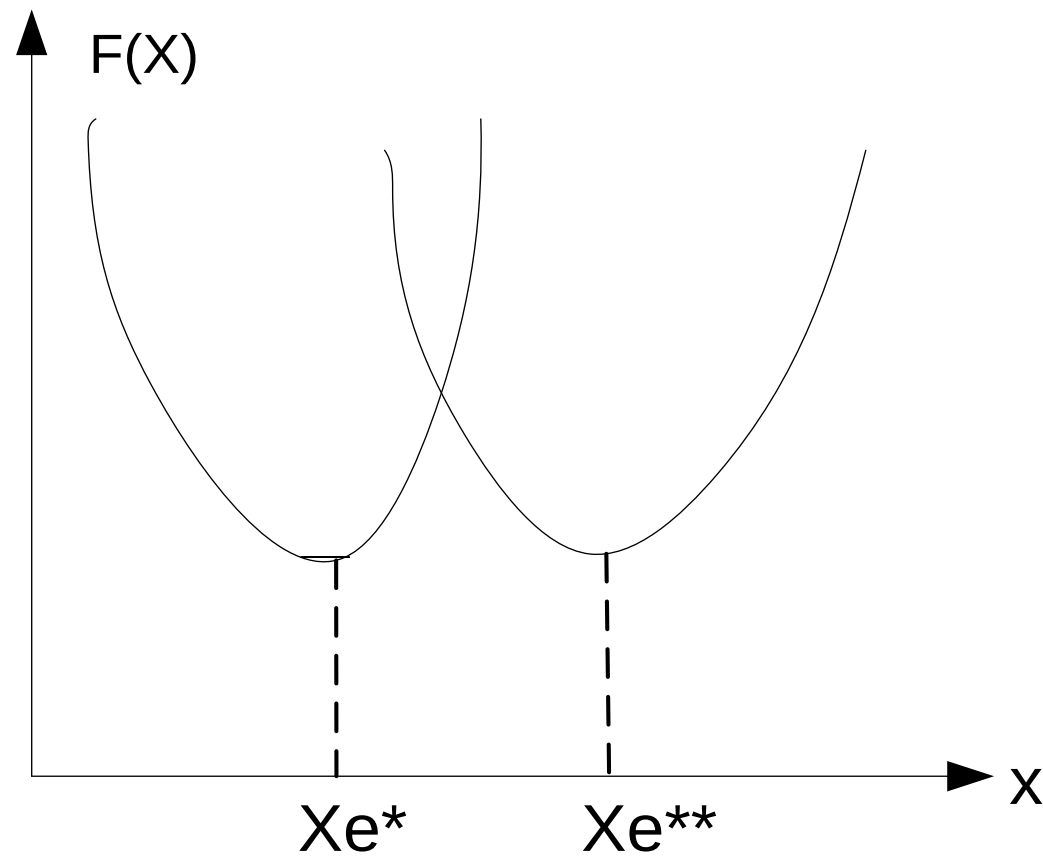
Много экстремумов

Виды задач:

методы многоэкстремальной
оптимизации (много
экстремумов — полимодальная
функция);

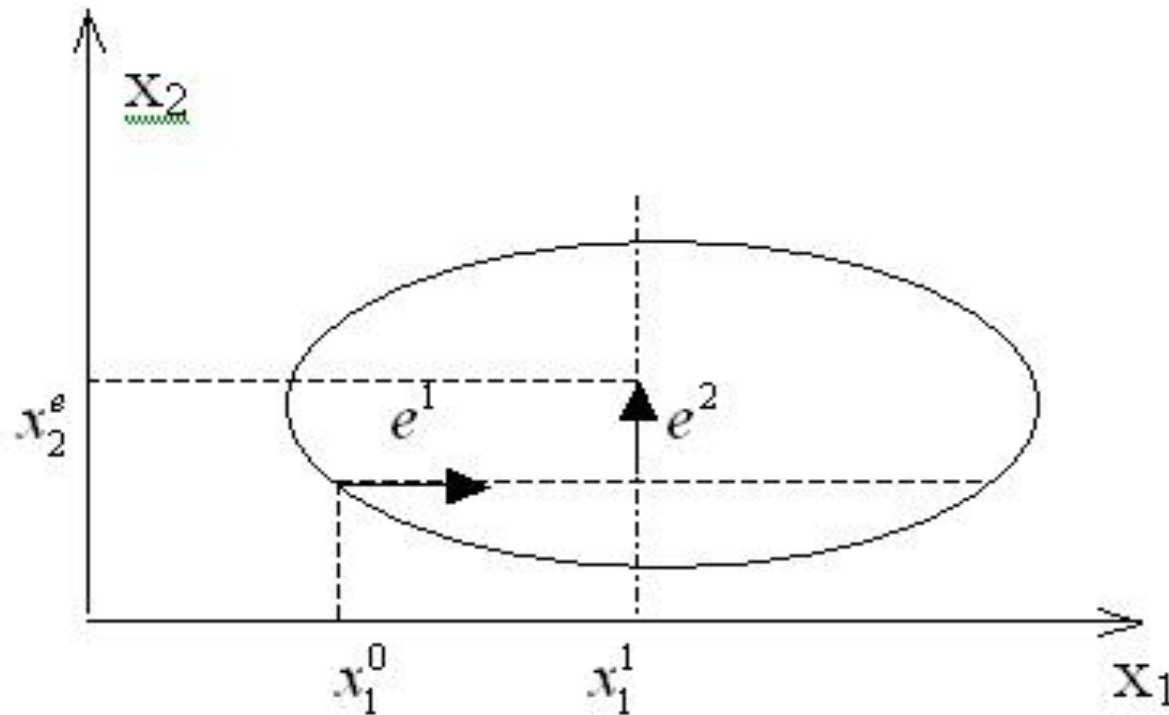


Многокритериальная оптимизация



Метод покоординатного спуска (метод 0-го порядка)

Функция $\varphi(x) \in C^0(X)$ – непрерывная функция.



Эффективность численных методов ОПТИМИЗАЦИИ

В настоящее время не существует универсальных методов решения задач оптимизации.

Невозможно определить наилучший метод решения некоторой задачи $\mathcal{P}$. Однако возможно определить класс методов для определенного класса задач. Т.е. класс задач $\mathcal{F}$, где $\mathcal{P} \subset \mathcal{F}$ и $\mathcal{F}$ - выделенный класс задач. Обычно численные методы разрабатываются для решения многих однотипных задач с близкими характеристиками. Поэтому эффективность метода $\mathcal{M}$ на всем классе задач $\mathcal{F}$ можно считать естественной характеристикой его качества.

При этом следует учитывать, что метод $\mathcal{M}$ не имеет полной информации о решаемой задаче.

Теорема о «бесплатных завтраках»

- Теорема Дэвида Волперта и Уильяма Макриди о бесплатном обеде (иногда во множественном числе) “No Free Lunch” (NFL) появляется в 1997 году в "No Free Lunch Theorems for Optimization». Название отсылает к поговорке «Бесплатных завтраков не бывает», то есть нет простых путей к успеху.

- **Формулировка теоремы**

Если алгоритм хорошо работает над определенным классом задач, то он обязательно платит за это снижением производительности на множестве всех оставшихся задач.

Эффективность численных методов ОПТИМИЗАЦИИ

Заранее известная численному методу «часть» задачи $\mathcal{P}$ называется моделью решаемой задачи.

Обозначим модель решаемой задачи символом Σ . Обычно в модель включаются формулировка задачи, описание свойств функциональных компонент и т. д.

Для того чтобы распознать задачу $\mathcal{P}$ среди всех прочих задач из данного класса (и тем самым решить ее), численный метод должен уметь накапливать специфическую информацию о решаемой задаче. Этот процесс удобно описывать с помощью понятия оракула. Оракул $\mathcal{O}$ проще всего представить в виде некоторого устройства, которое отвечает на последовательные вопросы численного метода.

Эффективность численных методов оптимизации

Естественно определить эффективность метода $\mathcal{M}$ на паре $(\Sigma, \mathcal{O})$ как его эффективность на наихудшем представителе $\mathcal{P}_w$ из $(\Sigma, \mathcal{O})$. Заметим, что задача $\mathcal{P}_w$ может быть трудной только для этого конкретного метода $\mathcal{M}$

Эффективность метода $\mathcal{M}$ на задаче $\mathcal{P}$ определяется через общие вычислительные затраты метода $\mathcal{M}$, необходимые для того, чтобы решить задачу $\mathcal{P}$.

Решить задачу $\mathcal{P}$ означает найти ее приближенное решение с заранее заданной точностью $\varepsilon > 0$.

Введем обозначение $\mathcal{T}_\varepsilon$ для некоторого критерия остановки, способного оценить качество предлагаемого решения.

Эффективность численных методов оптимизации

Формальное определение класса решаемых задач

$$\mathcal{F} = (\Sigma, \mathcal{O}, \mathcal{T}_\varepsilon).$$

Для решения конкретной задачи $\mathcal{P} \in \mathcal{F}$ естественно применить некую *итеративную процедуру*. Именно в таком виде удобно записывать любой метод $\mathcal{M}$, работающий с оракулом $\mathcal{O}$.

Эффективность численных методов оптимизации

Общая итеративная схема

Вводные данные: начальная точка x_0 и требуемая точность $\varepsilon > 0$.

Настройка. Полагаем $k = 0$ и $I_{-1} = \emptyset$. Здесь k — это счетчик итераций, а I_k — это накапливаемая информационная модель решаемой задачи.

Основной цикл.

Шаг 1. Задаем вопрос оракулу $\mathcal{O}$ в точке x_k .

Шаг 2. Пересчитываем информационную модель: $I_k = I_{k-1} \cup (x_k, \mathcal{O}(x_k))$.

Шаг 3. Применяем правила метода $\mathcal{M}$ для анализа модели I_k и формируем точку x_{k+1} .

Шаг 4. Проверяем критерий остановки $\mathcal{T}_\varepsilon$. Если ответ положительный, то генерируем ответ $\bar{x}$. В противном случае полагаем $k := k + 1$ и переходим на шаг 1.

Эффективность численных методов оптимизации

Наиболее трудоемкими являются два шага. Первый — это обращение к оракулу $\mathcal{O}$ Шаг 1. Второй Шаг 3, на котором анализируется накопленная модель и формируется следующая тестовая точка.

Таким образом, можно ввести две меры сложности задачи $\mathcal{P}$ и для метода $\mathcal{M}$.

Аналитическая сложность. Это число обращений к оракулу, необходимое для решения задачи $\mathcal{P}$ с точностью ε .

Арифметическая сложность. Это общее число всех вычислений (включая как работу оракула, так и работу метода), необходимых для решения задачи $\mathcal{P}$ с точностью ε .

Эффективность численных методов оптимизации

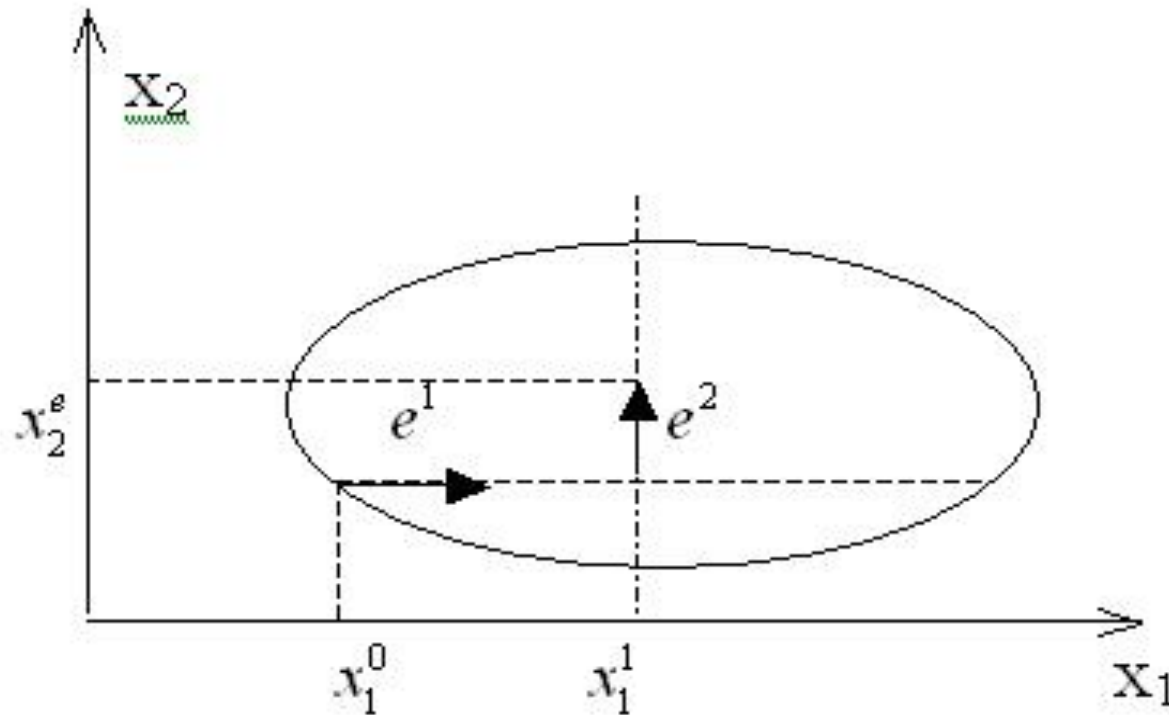
В соответствии со степенью гладкости функциональной модели можно пользоваться разными типами оракулов.

- Оракул *нулевого порядка*: возвращает значение функции $f(x)$.
- Оракул *первого порядка*: возвращает значение функции $f(x)$ и ее градиент $f'(x)$.
- Оракул *второго порядка*: возвращает $f(x)$, ее градиент $f'(x)$ и матрицу гессиана $f''(x)$.

Мы рассмотрим применение этих оракулов в различных методах.

Метод покоординатного спуска (метод 0-го порядка)

Функция $\varphi(x) \in C^0(X)$ – непрерывная функция.



Алгоритм:

Ш1. Направление d^k выбирается вдоль координаты x_k от точки $x^{(k)}$, т.е.

$$d^{(k)} = \frac{x_k}{\|x_k\|} = e^k$$

Ш2. Определяется величина шага

$$\alpha^{*(k)} = \operatorname{Arg} \min_{\alpha \in E^1} \varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$$

Ш3. Формируется новое значение

$$x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)}$$

Ш4. Проверка сходимости:

Если $(|\varphi(x^{(k+1)}) - \varphi(x^{(k)})| \leq \varepsilon, \|x^{(k+1)} - x^{(k)}\| \leq \delta)$, то *КОНЕЦ*

Иначе $k=k+1$ и переход на Ш 1.

Достоинства: возможность оптимизации для задач с недифференцируемыми функциями.

Недостатки:

1. Низкая скорость сходимости (сублинейная, т.е. $\gamma < 1$)

2. Возможность закливания при плохой обусловленности задачи, т.е. когда функция $\varphi(x)$ имеет овражный характер.

Обусловленность задачи

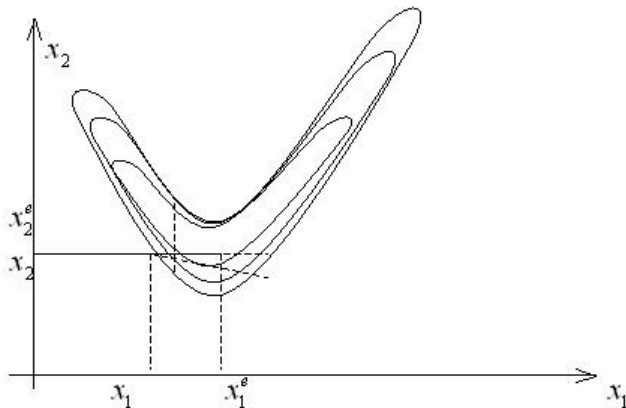
Определяется числом обусловленности матрицы Гессе (если она существует), т.е. отношением $\chi = \frac{L}{l}$.

Где L - максимальное собственное число $H(x^{(k)})$

l – минимальное собственное число $H(x^{(k)})$.

В случае отсутствия матрицы Гессе числом обусловленности может быть отношение:

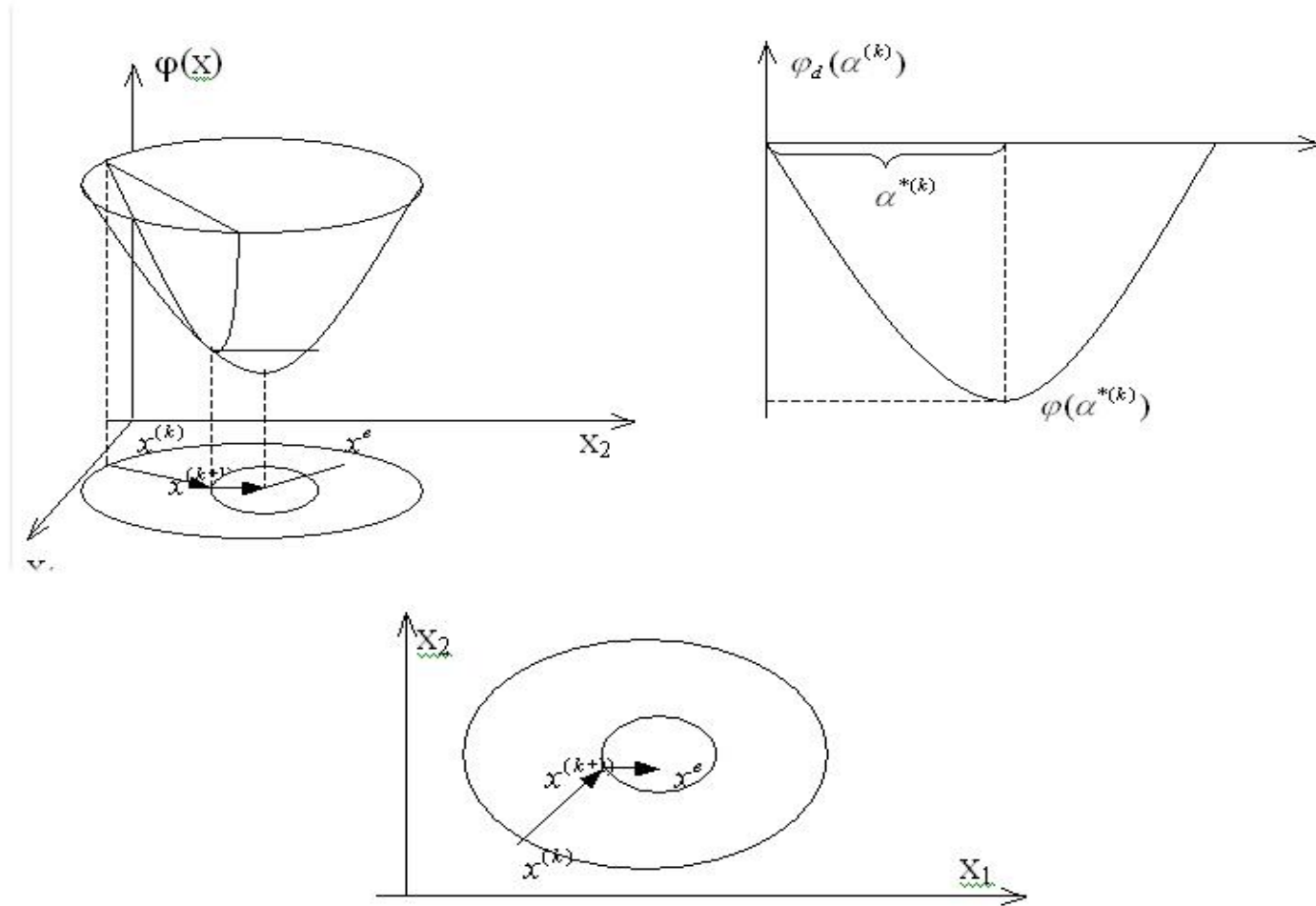
$$\chi = \lim_{\delta \rightarrow 0} \frac{\sup \|x^{(k)} - x^e\|^2}{\inf \|x^{(k)} - x^e\|^2} .$$



Пример функция Розенброка :

$$f(x_1, x_2) = 100(x_2^2 - x_1)^2 + (x_2 - 1)^2$$

Метод наискорейшего спуска (метод 1-го порядка).



Метод наискорейшего спуска (метод 1-го порядка).

Функция $\varphi(x) \in C^1(X)$ - класс 1 раз дифференцируемых функций.

$x^{(k+1)} = x^{(k)} + \Delta x^{(k)} = x^{(k)} + \alpha^{(k)} d^{(k)}$ – в ряд Тейлора, тогда, если учитывать
Разложим функцию $\varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$ только 2 члена ряда, получим:

$$\varphi(x^{(k+1)}) = \varphi(x^{(k)}) + \nabla^T \varphi(x^{(k)}) \Delta x^{(k)} + o(\|\Delta x^{(k)}\|)$$

Для того, чтобы функция уменьшалась, необходимо, чтобы $d^{(k)} \sim -\nabla \varphi(x^{(k)})$
 $d^{(k)} = -\frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|}$. Тогда $x^{(k+1)} = x^{(k)} - \frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|} \alpha^{*(k)}$

При этом $\alpha^{*(k)} = \text{Arg min}_{\alpha \in E^1} \varphi(x^{(k)} + \alpha^{*(k)} d^{(k)})$

Особенность МНС: $\nabla^T \varphi(x^{(k+1)}) \nabla \varphi(x^{(k)}) = 0$; то есть $d^{(k+1)T} d^{(k)} = 0$.

Достоинства. Линейная скорость сходимости для функций $\varphi(x)$ близких к квадратичным.

Недостатки:

1. Функция $\varphi(x) \in C^1(X)$ – должна быть дифференцируема.
2. Возможность закливания при плохой обусловленности задачи.

Метод Ньютона (метод второго порядка)

Условием использования этого метода является:

$$\varphi(x) \in C^2(X),$$

а также условие выпуклости $\varphi(x)$ в окрестности точки поиска.

Точка x^* — стационарная точка (точка локального экстремума) .

Пусть: $\varphi(x^*) = \min(\varphi(x))$ Тогда :

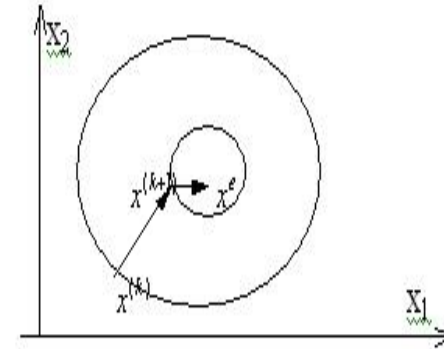
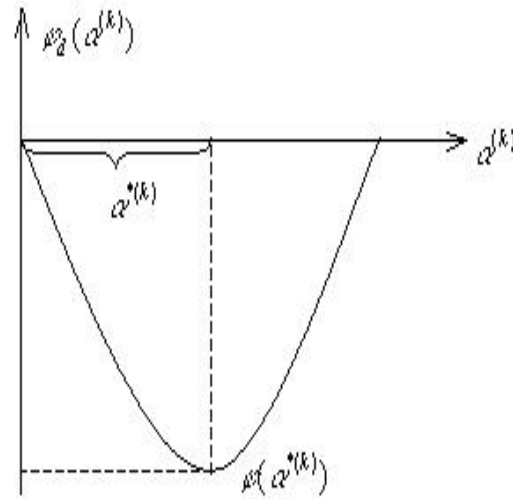
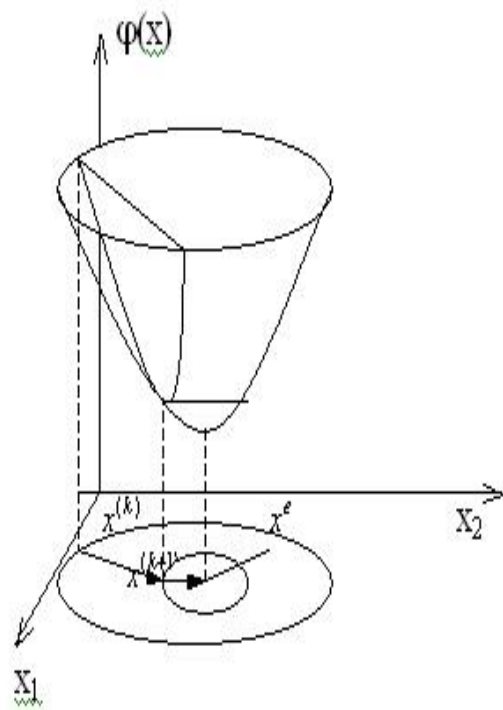
$$\bar{x} \in U_\varepsilon(x^*) \quad \varphi(x^*) = \varphi(\bar{x} + \Delta x) = \varphi(\bar{x}) + \nabla^T \varphi(x) \Delta x^* + \frac{1}{2} \Delta x^T H(\bar{x}) \Delta x + o(\|\Delta x\|^2)$$
$$x^* = \bar{x} + \Delta x \quad \text{где} \quad H(\bar{x}) = \left\{ \frac{\partial^2 \varphi(\bar{x})}{\partial x_i \partial x_j} \right\}_{i=1..n, j=1..n}$$

Разложение функции $\varphi(x^*)$ в окрестности x^* в ряд Тейлора до 3-го члена дает квадратичную аппроксимацию этой функции в $U_\varepsilon(x^*)$:

$$0 = \nabla \varphi(\bar{x}) + H^T(\bar{x})(x^* - \bar{x}) \quad . \quad x^{(k+1)} = x^{(k)} - H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}).$$

$$d^{(k)} = -H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}); \quad \alpha^{*(k)} = \text{Arg min } \varphi_d(\alpha^{(k)}); \quad x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)} .$$

Метод Ньютона (метод второго порядка)



Метод Ньютона (метод второго порядка).

Сопряженность: $d^{(k+1)} H^{-1}(x^{(k)}) d^{(k)} = 0$
Достоинства :

Для функций, близких к квадратичным, скорость сходимости алгоритма суперлинейная.

Недостатки:

- 1.Необходимость вычисления $H^{-1}(x^{(k)})$.
- 2.Процесс решения сильно зависит от выбора $x^{(0)}$ (т.е. от начальной точки)
- 3.Функция $\varphi(x)$ должна быть выпуклой во всех точках $x^{(k)}$, то есть $\det H(x^k) > 0$.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ

За последние 18-20 лет произошел огромный скачок в развитии систем искусственного интеллекта на основе машинного обучения и глубоких нейронных сетей. Это было обусловлено четырьмя основными факторами:

1. Значительным повышением производительности компьютерных систем.
2. Наличием доступных очень больших баз данных.
3. Созданием новых методов оптимизации на основе стохастического градиентного спуска.
4. Разработкой и созданием систем параллельной обработки информации на основе CUDA, GPU и TPU.

Мы рассмотрим те методы оптимизации, которые легли в основу машинного обучения и теории глубоких нейронных сетей.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ

Метод градиентного спуска является одним из наиболее популярных алгоритмов в области оптимизации для реализации обучения в современных нейронных сетях. В настоящее время разработан целый ряд библиотек глубокого машинного обучения, реализованных в виде соответствующих фреймворков. Ядром этих фреймворков являются методы оптимизации, основанные на использовании различных вариантов стохастического градиентного спуска.

Фреймворки для работы с глубокими сетями

1. Определение графа вычислений
2. Расчет графа вычислений
3. Дифференцирование графа

1. [TensorFlow](#) — это комплексная платформа для машинного обучения с открытым исходным кодом. Разработана компанией GOOGLE.

2. [PyTorch](#) — это среда машинного обучения на языке Python с открытым исходным кодом, обеспечивающая тензорные вычисления с GPU-ускорением. Она была разработана компанией Facebook и представлена в октябре 2016 года.

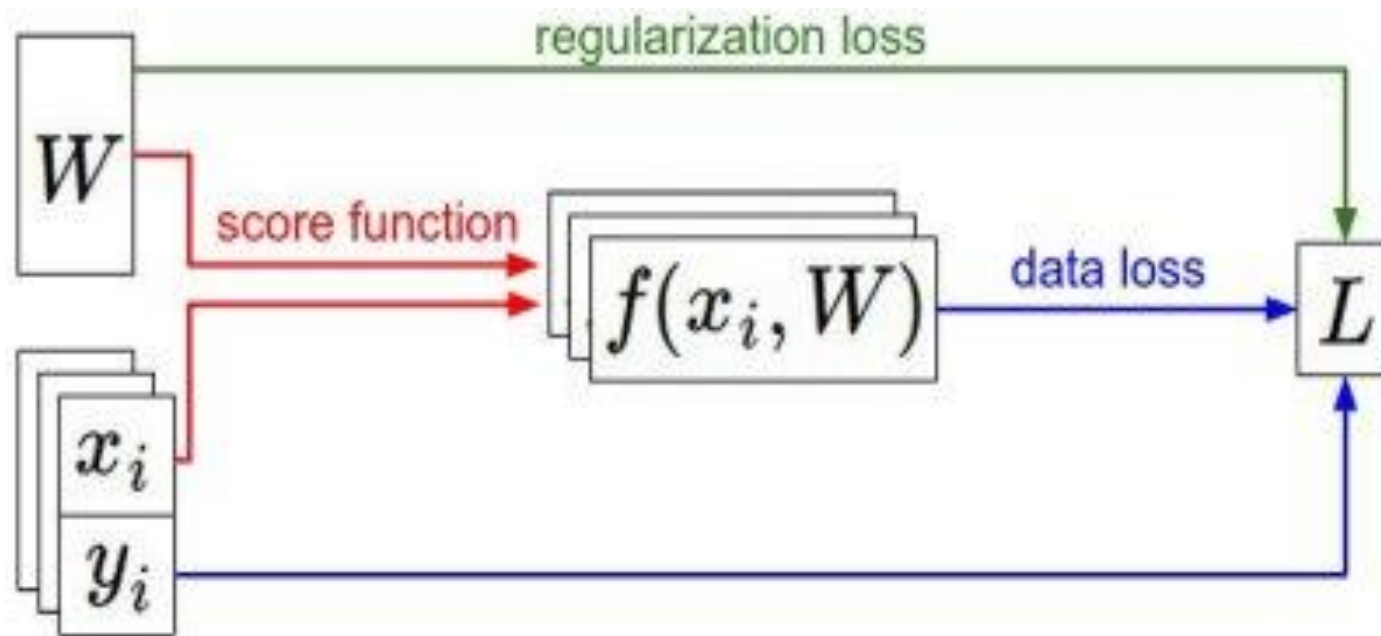
3. [Keras](#) — открытая среда глубокого обучения, написанная на Python. Она была разработана инженером из Google Франсуа Шолле и представлена в марте 2015 года.

4. [Darknet](#) — это фреймворк с открытым исходным кодом, написанный на языке C с использованием программно-аппаратной архитектуры параллельных вычислений CUDA.

5. [XGBoost](#) — это фреймворк с открытым исходным кодом, который предлагает систему градиентного бустинга для C++, Java, Python, R, Julia.

На следующей лекции мы рассмотрим основные концепции, касающиеся структуры и функционирования искусственных нейронных сетей.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ



ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Введем следующие обозначения:

θ — переменные оптимизации (веса синапсов искусственных нейронов, т.е. W в обозначениях на рисунке)

$\theta \in R^d$, где d — размерность вектора оптимизации.

$J(\theta)$ — целевая функция (функция ошибки или «потери»

$L(x, W, y)$ — в обозначениях на рисунке).

$\nabla_{\theta} J(\theta)$ — градиент целевой функции.

η — коэффициент обучения, определяющий величину шага в направлении локального минимума.

Выделим три основных варианта градиентного спуска.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Batch gradient descent.

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

*Градиентный метод, который рассчитан на вычисление градиента для **всего набора данных** (выборке - *batch*).*

Применение этого метода обладает следующими особенностями:

- 1) Требуется достаточно большого объема памяти.
- 2) Обладает низкой скоростью сходимости.
- 3) Низкой точностью.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Stochastic gradient descent (SGD)

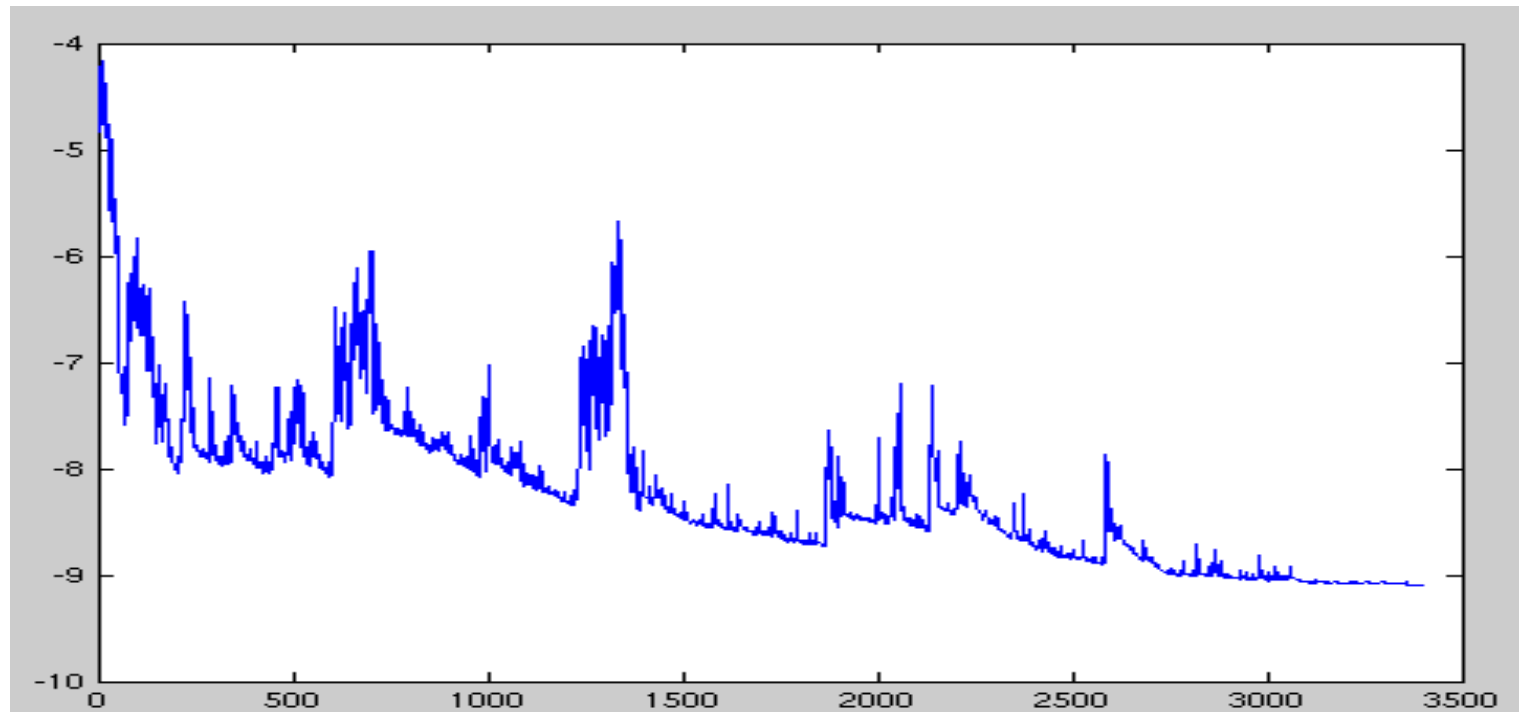
$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

*В отличие от традиционного градиентного метода, метод стохастического градиента позволяет обрабатывать за одну итерацию относительно небольшой набор обучающих данных из всего набора данных (выборки - *batch*).*

Применение этого метода позволяет избежать недостатки традиционного градиентного метода:

- 1) Требуется значительно меньше необходимого объема памяти.
- 2) Повышается скорость сходимости.
- 3) Увеличивается робастность к невыпуклости целевых функций.
- 4) Возможна оптимизация многоэкстремальных целевых функций.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ



Флуктуации целевой функции в процессе работы алгоритма

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Mini-batch gradient descent

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$$

*За каждую эпоху отбирается n данных ($n = 50 - 250$) небольшой набор обучающих данных из всего набора данных (выборки - *batch*).*

Применение этого метода дает следующие преимущества:

- 1) Уменьшает дисперсию разброса параметров что ведет к более устойчивой сходимости.
- 2) Позволяет использовать матричные и тензорные операции, входящие в состав современных библиотек методов глубокого обучения (смотри слайд 12).

Все это делает использование SGD на минибатчах очень эффективными для решения задач по глубокому обучению.

Далее мы рассмотрим дальнейшее развитие методов SGD.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Momentum

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$

Этот метод, применительно к SGD позволяет преодолевать овраги. Кроме того, он позволяет увеличить скорость сходимости алгоритмов SGD.

Метод моментов был предложен в 1964 году Б.Т. Поляком.

Другое название этого метода - «метод тяжелого шарика». Он рассматривался нами раньше в лекции посвященной многоэкстремальной оптимизации.

В 1986 году этот метод был использован Джеффри Хинтоном и Дэвидом Руммельхартом для обучения нейронных сетей методом обратного распространения ошибки.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Адаптивные методы

- 1. NAG. (Nesterov accelerated gradient)**
- 2. Adagrad. (Adapted gradient)**
- 3. Adadelat. (Extension of Adagrad)**
- 4. RMSProp. (Adaptive Root Mean Squared)**
- 5. Adam. (Adaptive moment estimation)**
- 6. AdaMax. (Modification Adam)**
- 7. Nadam. (Nesterov modification Adam)**

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

NAG. (Nesterov accelerated gradient)

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$

$$\theta = \theta - v_t$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adagrad. (Adapted gradient)

$$g_{t,i} = \nabla_{\theta_t} J(\theta_{t,i})$$
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}} g_{t,i}$$

$G_{t,ii}$ - диагональная матрица, содержащая l^2 -норму градиентов всех предыдущих итераций, включая текущую итерацию, для каждого измерения. Матрица G_t имеет следующий вид:

$$G_t = \text{diag} \left[\sqrt{\sum_{r=1}^i \theta_{r,t}^2 + \varepsilon} \right]_{t,i}$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adadelta. (Extension of Adagrad)

$$\theta_{t+1} = \theta_t + \Delta\theta_t.$$
$$\Delta\theta_t = -\frac{RMS[\Delta\theta_{t-1}]}{RMS[g_t]} g_t.$$

$$M[\Delta\theta_t^2] = \gamma M[\Delta\theta_{t-1}^2] + (1 - \gamma) \Delta\theta_t^2.$$

$$M[g_t^2] = \gamma M[g_{t-1}^2] + (1 - \gamma) g_t^2.$$

$$RMS[\Delta\theta_t] = \sqrt{M[\Delta\theta_t^2] + \varepsilon} - \text{среднеквадратичное значение переменной.}$$

$$RMS[g_t] = \sqrt{M[g_t^2] + \varepsilon} - \text{среднеквадратичное значение градиента.}$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

RMSProp. (Adaptive Root Mean Squared)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{M[g_t^2] + \varepsilon}} g_t.$$

$$M[g_t^2] = 0.9M[g_{t-1}^2] + 0.1 g_t^2.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adam. (Adaptive moment estimation)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t \quad .$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\beta_1 = \beta_2 = 0.98.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

AdaMax. (Modification Adam)

$$\theta_{t+1} = \theta_t - \frac{\eta}{u_t} \hat{m}_t.$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$u_t = \max(\beta_2 v_{t-1}, |g_t|)$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Nadam. (Nesterov modification Adam)

Является комбинацией NAG и Adam

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \left(\beta_1 \widehat{m}_t + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t} \right).$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$